Garment Retrieval with a From-Scratch Vision Transformer

Supervised Contrastive Learning for Render-to-Photo Matching in a 3D Clothing Pipeline

Computer Vision Team

Internal technical whitepaper - Computer Vision / 3D Assets

Abstract. We present Outfit Matcher, a garment-retrieval system that maps a photograph of a dressed person to the exact matching garments in a 3D render catalog. The visual encoder is a Vision Transformer (ViT-S/16, 21.9M parameters) implemented and trained entirely from scratch - no pretrained weights are used at any stage. Training follows the supervised contrastive (SupCon) paradigm with multi-view positives: the four canonical render angles of one garment are treated as positive pairs, which pulls all views of a garment onto a single point of the 256-dimensional unit hypersphere while pushing distinct garments apart. A key challenge is the domain gap between clean catalog renders (T-posed garment on white) and real photographs; we address it with heavy domain randomization - background keying via border flood fill, procedural background composites, perspective and affine warps, occlusion, blur, and noise. At inference, catalog views are embedded once and a query photo is matched by cosine similarity with per-garment max-pooling over views, yielding a ranked top-K list in milliseconds. We describe the architecture, mathematics, training objective, data pipeline, distributed two-GPU optimization, evaluation protocol (leave-one-view-out Recall@K and MRR with garment-level holdout), and full operational usage.

1 Introduction and Problem Statement

Our 3D pipeline composes garments combinatorially: garments are authored as separate meshes rather than single fused outfits. Ten shirts, ten pairs of pants, and ten hats already span one thousand unique outfits. Today, finding which catalog garments match a given reference photograph of a dressed person is a manual, slow, error-prone lookup task performed by artists. We automate it with learned visual retrieval.

$$\text{nts} \times 10\,\text{hats} = 1000\ \text{outfits, one garment at a time: } f_\theta(\text{photo}) \rightarrow$$

Formally: given a query image q containing a person wearing an unknown shirt, and a catalog C of N garments where each garment c is described by K = 4 canonical render views (front, back, left, right), output a ranking of catalog garments by visual match to q. This is an image-retrieval problem, not a classification problem: the catalog is dynamic (garments are added continuously), so the model must generalize to garments it has never seen and match by appearance similarity rather than memorized identity. Metric learning with a contrastive objective is the standard, well-matched tool for this setting: the encoder is trained to place same-garment views close together and different-garment views far apart in a metric space, after which nearest-neighbor lookup over precomputed catalog embeddings solves the ranking step without any retraining when the catalog changes.

2 Approach Overview

The system has three stages. (1) Offline catalog encoding: each garment's four render views are embedded once with the trained encoder and cached. (2) Query encoding: a photograph is embedded with the same encoder. (3) Matching: cosine similarities between the query embedding and all catalog view embeddings are computed; per-garment scores use the maximum over that garment's views; the top-K garments are returned with scores. Because both encodings live on the unit hypersphere, cosine similarity is a dot product, and the whole match step is a single matrix multiplication - milliseconds for catalogs of thousands of garments on CPU, and directly portable to GPU vector indexes (e.g. Faiss) at tens of millions of items.

2.1 System diagram

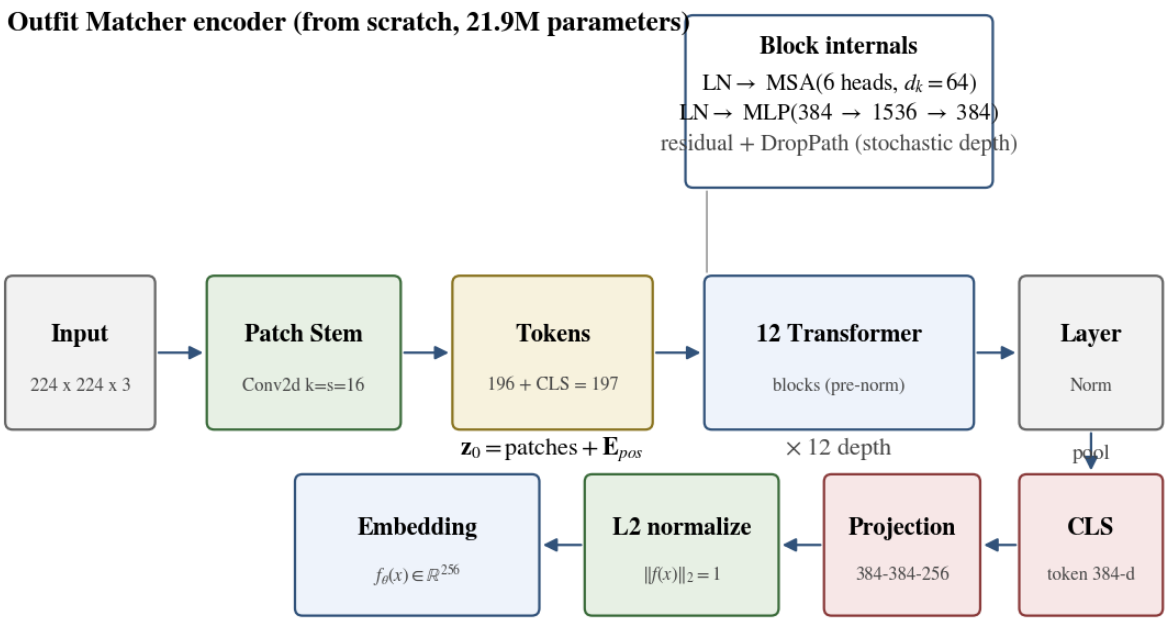

Figure 1: Outfit Matcher encoder architecture. Every component is trained from random initialization - no pretrained weights are used anywhere.

3 The Vision Transformer, from Scratch

The encoder is a standard ViT-S/16. An image is divided into a 14 x 14 grid of non-overlapping 16 x 16 patches; each patch is linearly embedded into a 384-dimensional token by a convolutional stem. A learnable classification token (CLS) is prepended, and a learnable absolute positional embedding is added so that the transformer, which is permutation-invariant over tokens, can reason about spatial layout. Twelve pre-norm transformer blocks with six attention heads and MLP ratio 4 process the 197-token sequence. The final LayerNorm output's CLS token serves as the global image representation. All weights are initialized from scratch (truncated normal, std 0.02) - the point of this project is that nothing is imported from a pretrained checkpoint.

3.1 Patch embedding

$$\mathbf{x} \in \mathbb{R}^{3\times224\times224} \Rightarrow \mathbf{X}_p \in \mathbb{R}^{196\times384}$$

Equation 1: Convolutional patchification. A Conv2d with kernel = stride = 16 maps each 16x16 patch to a 384-d token.

3.2 Token assembly and positional information

$$0 = [\,\mathbf{x}_{\text{cls}}\,;\,\mathbf{x}_p^1\mathbf{E}\,;\;\ldots\;;\,\mathbf{x}_p^N\mathbf{E}\,] + \mathbf{E}_{pos},\quad \mathbf{E}_{pos} \in \mathbb{R}^{(1+196)\times 38}$$

Equation 2: Input token sequence. A learnable CLS token is prepended; a learnable absolute positional embedding is added.

3.3 Multi-head self-attention

$$\mathrm{d}_i = \mathrm{softmax}\left(\frac{\mathbf{Q}_i\mathbf{K}_i^\top}{\sqrt{d_k}}\right)\mathbf{V}_i,\quad \mathbf{Q}=\mathbf{Z}\mathbf{W}^Q,\ \mathbf{K}=\mathbf{Z}\mathbf{W}^K,\ \mathbf{V}=\mathbf{Z}$$

Equation 3: Scaled dot-product attention per head (d_k = 64).

$$\mathrm{MHA}(\mathbf{Z}) = \mathrm{concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h)\,\mathbf{W}^O$$

Equation 4: Multi-head concatenation and output projection (h = 6 heads).

Self-attention lets every token gather information from every other token; heads specialize in different relations (local texture, hem-to-sleeve correspondence, global shape). With 197 tokens, full attention costs O(N^2) ~ 38,809 pairwise scores per head - cheap at ViT-S scale and fully parallel on GPU.

3.4 Transformer block (pre-norm residual)

$$\mathrm{opPath}(\mathrm{MHA}(\mathrm{LN}(\mathbf{z}_\ell))), \qquad \mathbf{z}_{\ell+1} = \mathbf{z}'_\ell + \mathrm{DropPath}(\mathrm{M}$$

Equation 5: Pre-norm block. LayerNorm before each sublayer; residual streams carry information; DropPath adds stochastic depth.

$$\mathbf{W}_2\,\mathrm{GELU}(\mathbf{W}_1\mathbf{u} + \mathbf{b}_1) + \mathbf{b}_2, \quad \mathbf{W}_1 \in \mathbb{R}^{1536\times 384}, \ \mathbf{W}_2 \in$$

Equation 6: MLP with GELU.

Pre-norm (LayerNorm inside the residual branch, not on the trunk) is the modern default: it keeps the residual stream clean, which stabilizes from-scratch training at high learning rates without LayerNorm reparameterization or careful warmup tricks. The MLP expands 384 to 1536 (ratio 4) with GELU nonlinearity. Stochastic depth (DropPath) randomly drops entire residual branches per sample during training, which acts as an ensemble regularizer and is scheduled linearly from 0 (first block) to 0.1 (last block).

$$\overline{\mathrm{Path}}(a) = \frac{\mathbf{m}}{1-p}\,a, \quad \mathbf{m} \sim \mathrm{Bernoulli}(1-p), \quad p_\ell = 0 + (\ell/11$$

Equation 7: Stochastic depth (per-sample), linearly scheduled from 0 to 0.1 across the 12 blocks.

3.5 Projection head

$$f_\theta(x) = \mathrm{normalize}(\mathbf{W}_2\,\mathrm{GELU}(\mathbf{W}_1\,\mathbf{z}_{L,0} + \mathbf{b}_1) + \mathbf{b}_2) \in \mathbb{S}^{25}$$

Equation 8: Two-layer MLP projection head to the 256-d hypersphere S^255 (L2-normalized output).

Contrastive methods consistently improve when the loss operates on a projected representation rather than the raw backbone features (SimCLR, SupCon). We use a two-layer MLP head (384 to 384, GELU, 384 to 256) whose output is L2-normalized onto the unit hypersphere S^255. Retrieval at inference uses the same normalized head output, so training and deployment geometries are identical.

Component	Shape	Parameters
Patch stem (Conv2d)	384 x 3 x 16 x 16	294,912
Positional embedding	197 x 384	75,648
CLS token	1 x 384	384
12 x Block (MHA + MLP)	12 x (1.81M)	21,692,160
Final LayerNorm	384	768
Projection head	384-384-256 MLP	364,800
Total		22,341,392 (backbone 21.9M)

Table 1: Parameter inventory. Backbone 21.9M; head 0.36M. All initialized from scratch (trunc-normal std 0.02).

4 Training Objective: Supervised Contrastive Learning

In each training batch we place P garments with V views each (defaults: P = 32 per GPU, V = 2, global batch 256 across two GPUs). Each view is independently augmented, so two views of the same garment are two different-looking images. The supervised contrastive loss then operates on the normalized embeddings z_i:

$$\mathcal{L}_{\text{SupCon}} = \sum_{i \in I} \frac{-1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(\mathbf{z}_i \cdot \mathbf{z}_p / \tau)}{\sum_{a \in A(i)} \exp(\mathbf{z}_i \cdot \mathbf{z}_a / \tau)}$$

Equation 9: Supervised contrastive loss over the multi-view batch (Khosla et al., 2020).

$$A(i) = I \setminus \{i\} \text{ (all except self)}, \quad P(i) = \{p \in A(i) : y_p = y_i\} \text{ (positives}$$

Intuitively: for every anchor, at least one other batch element is the same garment seen differently (augmented, different angle). The loss maximizes the log-probability, under a softmax over pairwise similarities, of those positives relative to all other elements. Same-garment views are pulled to a point; other garments are pushed away. Multi-view positives (Khosla et al. show 2+ views per class improves over single-positive InfoNCE) and in-batch negatives from the other 31 garments provide the gradient signal. Because views differ in angle and augmentation, the encoder cannot rely on view-specific cues - it must learn garment-invariant appearance, which is exactly what cross-view retrieval against real photos requires.

4.1 Auxiliary classification loss

$$\mathcal{L}_{\text{CE}} = -\sum_{i} \log \frac{\exp(\mathbf{g}_i^{(y_i)})}{\sum_{c=1}^{C} \exp(\mathbf{g}_i^{(c)})}, \quad \mathbf{g}_i = \mathbf{W}_{\text{cls}}\, \mathbf{z}_{L,0}^{(i)} \in \mathbb{R}^C$$

Equation 10: Auxiliary cross-entropy on the CLS token via a linear classifier (one logit per training garment).

$$\mathcal{L} = \mathcal{L}_{\text{SupCon}} + 0.5\, \mathcal{L}_{\text{CE}}$$

Equation 11: Total loss.

4.2 Why these losses

Why SupCon alone is not enough: with few hundred to a thousand garments, pure metric learning can wander - all garments can collapse toward similar directions while still locally ordering neighbors. The auxiliary linear classifier on the CLS token adds an explicit identity signal: features must also be linearly separable per training garment, which sharpens the space. Empirically in the contrastive-literature this combination (metric + auxiliary CE) is a robust default; our weight of 0.5 keeps SupCon dominant. The classifier head is discarded at inference.

4.3 What the embedding space looks like

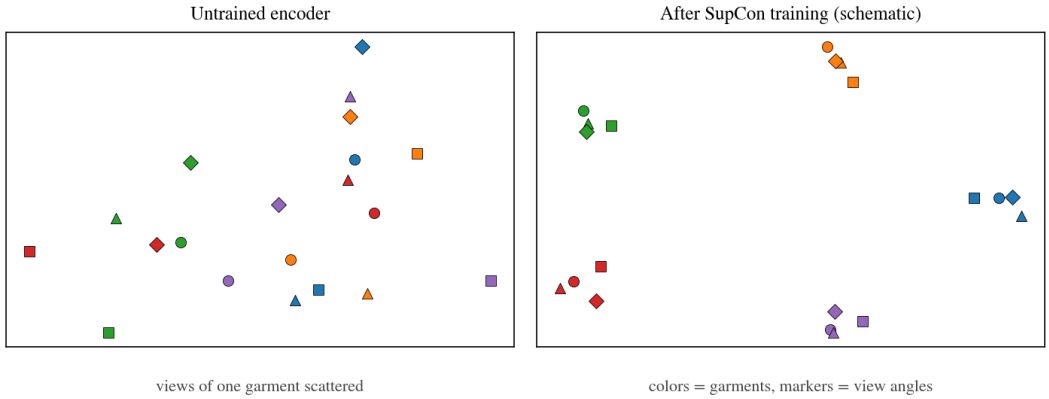

Figure 2: Supervised contrastive learning pulls all views of one garment together and pushes different garments apart, which is exactly the geometry nearest-neighbor retrieval needs.

5 Data Pipeline and Domain Randomization

Training data is the existing catalog renders: each garment directory contains front, back, left, and right views of the T-posed garment on a plain white background, without a human model. A preparation script scans the render tree, resolves angle names (front/f, back/b, left/l/side, right/r), and writes JSONL manifests. Garments are split by identity (default 5% held out): held-out garments never appear in training, so evaluation measures generalization to unseen garments, which mirrors production.

5.1 Augmentation pipeline

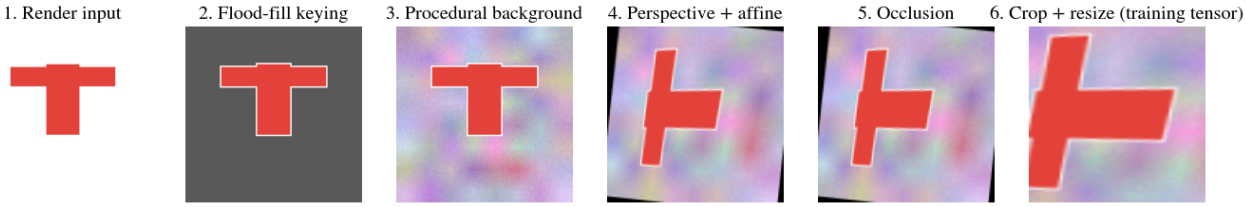

Figure 3: Actual pipeline stages executed by the shipped augmentation code (outfit_matcher/data/transforms.py) on a sample render: keying, procedural background, perspective + affine warp, occlusion, crop.

Augmentation	Probability	Parameters
Background keying (flood fill)	always (85% composite)	tol=0.08, grow=2px
Procedural background	0.85	8 palettes + low-freq noise
Perspective warp	0.3	corner shift up to 15%
Rotation + scale	always	+/-7 deg, 0.95-1.1
Color jitter	always	brightness/contrast/sat x0.4
Grayscale	0.05	-
Gaussian blur	0.2	sigma 0.5-1.5, k=3/5
Gaussian noise	0.2	sigma 0.01-0.05
Occluder rectangles	0.5	2-15% of image, up to 2
Garment crop	always	scale 0.5-1.0 of bbox
Horizontal flip	0.5	-

Table 2: Domain randomization inventory (probability = per-sample chance).

The renders are too clean: uniform background, perfect lighting, centered framing. A model trained on them verbatim will not transfer to photographs. Domain randomization closes this gap by training on synthetic disturbances of the renders (Tobin et al.): if the training distribution already spans backgrounds, lighting, viewpoints, and occlusions, the real-photo distribution falls inside it. Concretely (Figure 3, Table 2): the near-white background is keyed by flood fill from the image border (only border-connected white is removed, so white fabric interior is preserved); the garment is composited onto procedurally painted backgrounds; mild perspective and affine warps simulate off-axis cameras; photometric jitter, blur, and noise simulate consumer photography; random rectangles simulate arms and objects partially covering the subject; and scale-jittered crops around the garment bounding box simulate framing variation. Horizontal flips are applied - for garments this is safe and doubles data.

5.2 Balanced multi-view batching

SupCon needs positives in every batch, so batches are constructed by a balanced multi-view sampler rather than shuffled indices: each batch draws P garments, and for each garment samples V of its available views, guaranteeing every batch contains P positive groups of size V. In the two-GPU setup the garment set is sharded by rank so the two GPUs see different garments - the effective global batch has 2P garments and 2PV images, and the SupCon implementation gathers embeddings across ranks so negatives span both GPUs.

6 Optimization and Infrastructure

From-scratch ViTs need modern optimization to be stable: AdamW (decoupled weight decay, applied to weights but not biases or norm parameters), a peak learning rate around 1e-3 for batch 256, 5 epochs of linear warmup, and cosine decay to 1e-5. Gradients are clipped at global L2 norm 1.0. Mixed precision uses bfloat16 autocast (native on RTX 5090) with float32 loss computation. In addition, an exponential moving average of all weights (decay 0.999) is maintained; all evaluation and the exported checkpoint use EMA weights, which is a free, consistent accuracy gain.

$$-1+(1-\beta_1)\mathbf{g}_t,\quad \mathbf{v}_t=\beta_2\mathbf{v}_{t-1}+(1-\beta_2)\mathbf{g}_t^{\odot 2},\quad \mathbf{p}_t\leftarrow\mathbf{p}_{t-1}-\eta_t\left(\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t}+}\right.$$

Equation 12: AdamW with decoupled weight decay; bias-corrected moments.

$$\eta_t=\eta_{\min}+\frac{1}{2}(\eta_{\max}-\eta_{\min})\left(1+\cos(\pi\frac{t-t_w}{T-t_w})\right),\quad t<t_w:\ \eta_t=\eta_{\max}\,t/t_w$$

Equation 13: Cosine schedule with 5-epoch linear warmup (eta_max = 1e-3, eta_min = 1e-5).

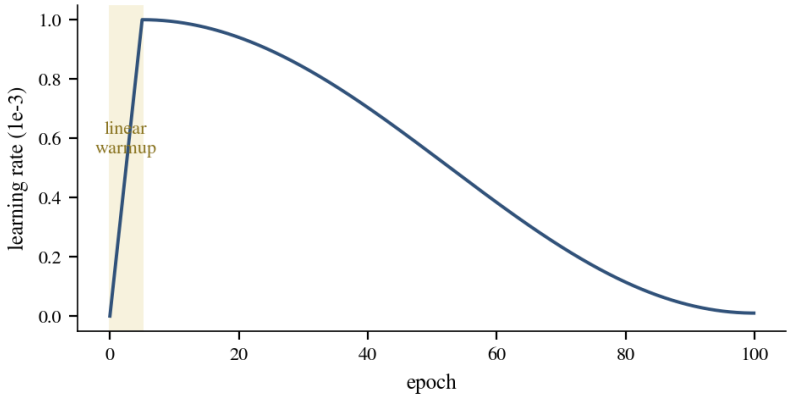

Figure 4: Learning-rate schedule over 100 epochs.

$$\theta_{\text{EMA}}\leftarrow m\,\theta_{\text{EMA}}+(1-m)\,\theta,\qquad m=0.999$$

Equation 14: Exponential moving average of weights used for all evaluation and export.

Hyperparameter	Value	Rationale
Optimizer	AdamW	decoupled weight decay, ViT standard
Base LR	1e-3	scaled for batch 256
Min LR	1e-5	cosine floor
Warmup	5 epochs (linear)	stabilizes early attention
Schedule	cosine	smooth decay to floor
Weight decay	0.05	on weights only; 0 on bias/norm
Batch (global)	256 = 2 GPUs x 32 garments x 2 views	balanced multi-view SupCon
Temperature tau	0.1	sharpens softmax over sims
SupCon + aux CE	1.0 : 0.5	retrieval + classification signal
DropPath	0.0 to 0.1 (linear)	stochastic depth regularization
EMA decay	0.999	eval/export weights
Epochs	100	convergence on small datasets
Precision	bf16 autocast	RTX 5090 native

Hyperparameter	Value	Rationale
Grad clip	1.0 (global L2 norm)	exploding-gradient guard

Table 3: Training hyperparameters (configs/shirts.yaml).

6.1 Dual-GPU distributed training

Training runs on two RTX 5090 GPUs with PyTorch DistributedDataParallel. On Windows the NCCL backend is unavailable, so we use gloo. Two subtleties matter: (i) the SupCon loss gathers features and labels across ranks via all_gather (on gloo this round-trips through CPU tensors), with gradients flowing only through the local rank's block; (ii) the auxiliary classifier is not wrapped by DDP, so its gradients are manually all-reduced and averaged. The launcher is a small custom Python script (scripts/launch_ddp.py) that spawns one worker per GPU with MASTER_ADDR/MASTER_PORT/RANK/LOCAL_RANK environment variables rather than torchrun. Reason: several torch Windows builds (including 2.5.1) ship without libuv, and torchrun's elastic rendezvous then fails at TCPStore creation; init_process_group('env://') with USE_LIBUV=0 works correctly, so the custom launcher sidesteps the broken component. Each worker binds to its own GPU via LOCAL_RANK.

7 Evaluation Protocol and Metrics

Evaluation answers: for a garment never seen in training, does the model rank its other views correctly against every other held-out garment's views? All held-out garments' views are embedded. Each view takes a turn as query; the gallery is every other view (same-garment views excluded from the gallery only when the query is that garment's own duplicate - in practice each of the 4 angles queries against all views of all other held-out garments). A retrieval is correct when the top-ranked gallery view belongs to the same garment (necessarily a different angle, since the query view itself is excluded from its own gallery). We report Recall@1/5/10/20 and MRR. This protocol is strict: it measures cross-angle matching, generalization to unseen garments, and discrimination among near-duplicate garments simultaneously. A real-photo validation set (labeled photos matched to garment IDs) can be dropped in as an additional manifest; the code paths are identical.

$$\text{Recall@}K = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \mathbf{1}\left[\hat{c}(q) \text{ in top} - K \ \wedge \ \hat{c}(q) = c^{*}(q)\right]$$

Equation 15: Recall@K over the query set.

$$\text{MRR} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \frac{1}{\text{rank}_q}, \quad \text{rank}_q = \text{position of first correct garmen}$$

Equation 16: Mean reciprocal rank.

$$\text{ews}\,\}, \ \ \text{gallery} = \mathcal{Q} \setminus \{\text{query view}\} \ \wedge \ \text{correct} \Leftrightarrow \text{same garment, dif}$$

Equation 17: Leave-one-view-out protocol. A retrieval is correct only if the gallery view belongs to the same garment at a different angle.

8 Inference: Matching a Photo to the Catalog

The catalog is embedded once: each garment contributes its four (or fewer) view embeddings, L2-normalized, to a matrix. A query photo is preprocessed identically to training input statistics (resize, center-crop to 224) and embedded. Matching is a dot product against the catalog matrix. Per-garment scores take the max over that garment's views, because a photo typically resembles one canonical angle more than others, and garments are ranked by that best-view score. Output is a top-K list of garment IDs with cosine similarities.

$$\hat{c}(q) = \arg\max_{c \in \mathcal{C}} \ \max_{v \in \text{views}(c)} \frac{f_\theta(q) \cdot f_\theta(v)}{\|f_\theta(q)\| \, \|f_\theta(v)\|}$$

Equation 18: Match rule - the best-scoring view determines the garment.

$$\text{re}(q,c) = \max_{v \in \text{views}(c)} s(f_\theta(q), f_\theta(v)), \qquad s(u,w) = u \cdot w \text{ (unit vecto}$$

Equation 19: Per-garment score for top-K ranking (max over the garment's catalog views).

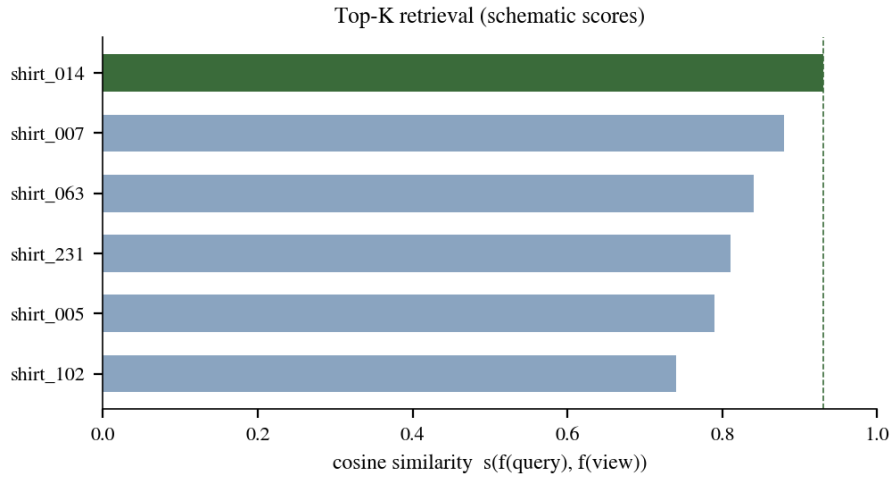

Figure 5: Top-K retrieval output. Scores are cosine similarities in [-1, 1].

8.1 Operational notes

Encoding the whole catalog takes seconds on GPU and the artifact is a single .pt file; re-embedding is only needed when the catalog changes (and can be incremental - append new garments' rows). Matching itself is one matrix product; for very large catalogs the matrix should move to Faiss or a vector database, with the same embeddings. Query preprocessing (resize + center-crop) matches eval, not the random training crops - intentionally, because real photos are already naturally framed. If a query contains multiple garments (full outfit), run the pipeline per detected garment region or per garment category classifier first; per-category models (shirts now, pants and hats later) keep each space clean, since a shirt should never be matched against pants.

9 How to Run It

9.1 Prepare manifests

```
cd outfit-matcher
pip install -r requirements.txt

python -m outfit_matcher.data.prepare_data ^
    --data-root D:\renders\shirts ^
    --out-dir  D:\data\shirts ^
    --val-fraction 0.05 --seed 42
```

9.2 Train on two GPUs

```
scripts\train_ddp.bat D:\data\shirts configs\shirts.yaml
:: equivalent: python scripts/launch_ddp.py --nproc 2 --config configs/shirts.yaml ^
::                                      --data-override D:/data/shirts
```

Checkpoints land in runs/shirts_v1: checkpoint_last.pt (resumable, includes optimizer and EMA state), periodic snapshots every 10 epochs, checkpoint_final.pt, and JSONL histories (loss per epoch; Recall@K/MRR every 5 epochs). Resume with train.resume in the YAML or --resume.

9.3 Match a photo

```
python -m outfit_matcher.match ^
    --config configs/shirts.yaml ^
    --checkpoint runs\shirts_v1\checkpoint_final.pt ^
    --catalog D:\data\shirts\manifest_train.jsonl ^
    --catalog-cache runs\shirts_v1\catalog_emb.pt ^
    --query C:\photos\someone.jpg ^
    --topk 5
```

9.4 Smoke tests

```
python -m pytest tests/ -q
:: generates a synthetic garment dataset, trains a tiny model end-to-end,
:: runs eval + retrieval + the prepare-data CLI. Green in ~1 min on CPU.
```

File	Role
outfit_matcher/model/vit.py	ViT backbone, projection head (no pretrained weights)
outfit_matcher/losses/supcon.py	SupCon loss with cross-GPU gathering
outfit_matcher/data/prepare_data.py	render tree -> manifests + train/val garment split
outfit_matcher/data/dataset.py	dataset + balanced multi-view batch sampler
outfit_matcher/data/transforms.py	domain randomization: keying, warps, occluders
outfit_matcher/engine.py	DDP init (gloo/Windows), EMA, AdamW groups, cosine schedule
outfit_matcher/train.py	training loop, checkpoints, history, eval hooks
outfit_matcher/evaluate.py	leave-one-view-out Recall@K / MRR + catalog encoding
outfit_matcher/match.py	production CLI: photo -> top-K catalog garments
scripts/launch_ddp.py	custom 2-GPU DDP launcher (torchrun-free, libuv-safe)
scripts/train_ddp.bat	one-click dual-5090 training entry

Table 4: Repository map.

10 Limitations and Future Work

First, the model has never seen a human body: renders are T-posed garments without a wearer, so sleeves drape differently in real photos. Heavy augmentation mitigates but does not remove this; the strongest fix is compositing garments onto our existing 3D human models at varied poses and re-rendering - the training pipeline already accepts any per-garment view images, so this is a data change, not a code change. Second, scale: 100-1000 garments is small for from-scratch training; the balanced sampler and strong regularization are chosen for it, but accuracy will scale with catalog size. Third, occlusion by other garments (jackets over shirts) is only simulated synthetically. Fourth, this first model is shirt-only by design; pants and hats should be trained as separate category models and, optionally, distilled later into one unified embedding space. Finally, evaluation on real photos is pending labeled data; the leave-one-view-out numbers measure render-side generalization only.

References

[1] Dosovitskiy, A., et al. (2020). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. ICLR 2021.

[2] Khosla, P., et al. (2020). Supervised Contrastive Learning. NeurIPS 2020.

[3] Chen, T., et al. (2020). A Simple Framework for Contrastive Learning of Visual Representations (SimCLR). ICML 2020.

[4] Tolstikhin, I., et al. (2021). MLP-Mixer. Appendix: from-scratch ViT training recipes (AdamW, high LR, strong augmentation).

[5] Huang, G., et al. (2016). Deep Networks with Stochastic Depth. ECCV 2016.

[6] Loshchilov, I., & Hutter, F. (2019). Decoupled Weight Decay Regularization (AdamW). ICLR 2019.

[7] He, K., et al. (2016). Deep Residual Learning. Residual connection formulation.

[8] Tobin, J., et al. (2017). Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World. IROS 2017.

[9] Oquab, M., et al. (2023). DINOv2: Learning Robust Visual Features without Supervision. EMA + strong augmentation practice.

[10] Johnson, J., Douze, M., & Jegou, H. (2019). Billion-scale similarity search with GPUs. IEEE TBDC. (Faiss, nearest-neighbor retrieval.)